Science of Computer Programming 78 (2013) 762–774

Contents lists available at SciVerse ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico

Multi-agent approach to distributed ant colony optimization

Sorin Ilie, Costin Bădică *

University of Craiova, Software Engineering Department, Bvd. Decebal 107, Craiova, 200440, Romania

ARTICLE INFO

Article history:
Received 17 March 2011
Received in revised form 13 September 2011
Accepted 15 September 2011
Available online 29 September 2011

Keywords:
Distributed multi-agent middleware
Ant colony optimization
Traveling salesman problem

ABSTRACT

This paper presents a configurable distributed architecture for ant colony optimization. We represent the problem environment as a distributed multi-agent system, and we reduce ant management to messages that are asynchronously exchanged between agents. The experimental setup allows the deployment of the system on computer clusters, as well as on ordinary computer networks. We present experimental results that we obtained by utilizing our system to solve nontrivial instances of the traveling salesman problem. The results support the effectiveness and scalability of our proposal.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

Collective natural bio-systems like ant and bee colonies, flocks of birds and swarms, as well as systems of cells and molecules, are composed of multiple bio-entities residing in the physical environment and engaged in complex collective and organized behaviors, interactions and processes according to the laws of nature. There is a certain level of abstraction at which the behavior of such systems can be modeled as distributed computational processes resulting from the interaction of artificial computational entities. Thus, we would expect distributed computing to have a lot of potential for the practical application of nature-inspired computing, i.e., computing inspired by the behavior of natural bio-systems.

In particular, ant colony optimization (ACO hereafter) [1] is inspired by the collective behavior of colonies of natural ants when they explore the environment searching for food. During their search process, ants secrete pheromone on their way back to their anthill. Other ants of the colony sense the pheromone and are attracted to the marked paths; the more pheromone that is deposited on a path, the more attractive that path becomes. The pheromone is volatile and disappears over time. Evaporation erases the pheromone on longer paths as well as on less interesting paths. Shorter paths are refreshed more quickly, thus having the chance of being more frequently explored. Intuitively, ants will converge towards the most efficient path due to the fact that it gets the strongest concentration of pheromone.

Based on our literature review [1–6] we found that most of the works related to distributed ACO do not provide an intuitive and straightforward mapping of ACO algorithms onto distributed computing systems. Rather, existing approaches rely on adapting the classical sequential algorithms for parallel and high-performance computing architectures without a careful and in-depth consideration of the intrinsic distributed nature of ACO.

In this context, we support the idea that ACO should allow a more straightforward mapping onto existing distributed architectures, including multi-agent system middleware. Therefore, to take advantage of the full potential of nature-inspired computational approaches, we have started an investigation of new distributed forms of ACO using state-of-the-art multi-agent technology. The focus of our research is to propose a better computational architecture and programming for the implementation of ACO algorithms utilizing available state-of-the-art distributed multi-agent middleware. We foresee two

* Corresponding author.
E-mail addresses: sorin.ilie@software.ucv.ro (S. Ilie), costin.badica@software.ucv.ro, cbadica@software.ucv.ro (C. Bădică).

0167-6423/$ – see front matter © 2011 Elsevier B.V. All rights reserved.
doi:10.1016/j.scico.2011.09.001

potential benefits of our work: (i) first, it contributes to the engineering of ACO algorithms using distributed computing architectures, with the potential of obtaining more scalable and efficient algorithms; (ii) second, as a byproduct, our work contributes to better understanding of new forms of agent-based distributed ACO. *In this setting, the goal of our paper is to introduce a novel and more intuitive way of distributing ant-based nature-inspired computational processes using state-of-the-art multi-agent middleware and to experimentally investigate the feasibility of this approach.*

In our previous work [7–9], we proposed a scalable multi-agent system architecture called ACODA (ant colony optimization on a distributed architecture) that allows the distributed, asynchronous and decentralized implementation of ACO. The novelty of this approach is that (i) the physical environment of the ants is conceptualized, represented and implemented as a distributed multi-agent system [10], and (ii) the movement of ants in the physical environment is represented as messages exchanged asynchronously between the agents of the problem environment. Note that, as ants are governed by similar behavioral rules, the representation of ants as mobile software agents cannot be motivated. Instead, messages were used, thus considerably simplifying the architecture and discarding the computational and communication overheads required by code mobility.

There are two main informal arguments that motivate the suitability of our approach for the distributed implementation of ACO algorithms. The first motivates the use of multi-agent systems for modeling the ants' environment. The second motivates the mapping of the ants' environment to a distributed architecture and the mapping of the ants' movements to messages exchanged between the agents located in the ants' environment.

First, although the behavior of natural ants can be described as complex, this complexity is mainly a consequence of the complexity of the physical environment in which the ants live and interact, rather than of the ants' intrinsic "intelligence". This can be noticed by carefully observing that ants' movement in their environment actually "mimics" the complexity of this environment, and therefore their emerging sequence of decisions and movements in searching for a path appears to be very complex and "intelligent" to the external observer. For example, in [11], the authors determine that the distribution of the ants is influenced more by the environment than by their interactions. Moreover, the authors of [12] compare the behavior of ants with chemical molecules under the influence of physical laws, that have no intelligence of their own, although they are able to exhibit intelligence by acting collectively in a physical environment. These observations support the idea of multi-agent modeling of the ants' environment as a complex system, while on the other hand they suggest the feasibility of keeping the ants as simple as messages exchanged between the agents.

Apparently, our approach suffers of interaction overhead between distributed agents. However, this interaction complexity can be at least partially compensated by the appropriate distribution of agents, as is explained in what follows. So, for the second motivation, let us consider an ACO problem in which the environment is represented as a distributed multi-agent system consisting of N agents. Intuitively, by distributing the agents on several machines, the agent communication load is divided between them, thus resulting in the execution time being reduced. To simplify our analysis, we assume that there is the same number Q of ant migrations between any two agents in the system. We further assume that the cost of exchanging an ant message on a single machine is equal to constant a, while the cost of exchanging an ant message between two different machines is $b > a$. In the extreme case when the system is mapped to a single machine, it follows that the execution time is at least $a * Q * N * (N - 1)/2$, i.e., the total cost required to exchange all the ant messages. However, at the other extreme, if we map each agent to a separate machine, then the communication cost of each machine is $b * Q * (N - 1)$. Now, if $N > 2 * b/a$, i.e., if the number of machines is sufficiently large, then the execution time of each machine will be reduced because its communication load is reduced, thus clearly supporting our "ants as messages" approach.

The paper is organized as follows. Section 2 introduces the basics of ACO with a focus on the ant colony system (ACS in what follows), as well as on the traveling salesman problem (TSP) that was used as a benchmark in this paper. In Section 3, we discuss the architecture of ACODA. In particular, we explain how a set of agents distributed on a computer network must behave and interact according to ACO rules to solve the TSP problem. Section 4 presents the evaluation of ACODA on TSP benchmark problems on computer networks equipped or not with a workload management system. Section 5 presents related works relevant for ACODA. In Section 6, we present our conclusions, and we outline future works.

2. Background

ACO refers to a family of optimization algorithms that get their inspiration from the metaphor of real ants searching for food. ACODA is a distributed architecture specially designed to support efficient implementation of distributed ACO algorithms.

The traveling salesman problem (TSP) [13] is often used to explore the usefulness and performance of ACO algorithms. The goal of the TSP is to compute a shortest tour that visits each node of a complete weighted graph exactly once. The decision version of TSP is NP-complete, which basically means that it is very unlikely that a polynomial solution for solving the TSP exists. So the TSP is a very good candidate for the application of heuristic approaches, including ACO. In our literature review, we noticed that other computational approaches for distributing ACO algorithms were applied to the TSP [2,5,3,4,14]; thus we adopted the TSP as a benchmark problem for our experiments (this is assumed throughout this paper).

The main idea behind ACODA is to provide an agent-based distributed infrastructure for modeling the ants' environment [15,10]. Artificial ants originating from the anthills that are located in the environment will travel to find optimal solutions, following ACO rules (see below). In order to use ACO to solve the TSP, the ants' environment is conceptualized as a distributed set of interconnected graph nodes that are managed by software agents (note however that this modeling is specific to the

TSP, so it must be updated when ACODA is applied to solving other problems). Ants travel between nodes following ACO rules until they find a solution, i.e., until they complete a tour for the TSP. Once they return to their originating anthill, they mark the solution with pheromone by tracing back their path. In ACODA, ant traveling is realized by messages exchanged asynchronously between the agents located in the problem environment. In [7], we used a single graph node (simply called node in what follows) per agent, while in this paper we will also investigate the effect of allocating more nodes to a single agent.

In our experiments, we used the distributed version of the ACS [1], initially proposed in [7]. The original ACS is a sequential implementation of ACO which chose to interleave the movement of ants rather than separately moving each ant until it finishes its tour. Our approach presents a few differences due to the requirements of a truly distributed architecture based on asynchronous message passing and the avoidance of using global knowledge, thus suiting the basic principles of ACODA.

In what follows, we briefly review the mathematical model of ACO that we employed as the basis for our system design and implementation.

At the core of ACO algorithms there are rules that determine the amount of pheromone deposited on edges traversed by ants, the edge chosen by each ant on its way, and how fast the deposited pheromone evaporates.

Ant a located at node i decides to move to node j with probability $p_{i,j}$ computed as follows:

$$p_{i,j} = \frac{(\tau_{i,j})^{\alpha}(\eta_{i,j})^{\beta}}{\Sigma_j(\tau_{i,j})^{\alpha}(\eta_{i,j})^{\beta}}, \tag{1}$$

where

- $\tau_{i,j}$ is the amount of pheromone deposited on edge (i, j),
- $\eta_{i,j}$ is the inverse of the weight of edge (i, j),
- α is a parameter to control the influence of $\tau_{i,j}$,
- β is a parameter to control the influence of $\eta_{i,j}$, and
- j represents a node reachable from node i that was not visited yet.

Better solutions need to be marked with more pheromone. So whenever ant a finds a tour V_a of cost L_a, better than the best known tour, the ant will increase the pheromone strength on each edge of the tour with a value $\Delta\tau^a_{i,j}$ that is proportional to the quality of the solution, i.e.,

$$\Delta\tau^a_{i,j} = \begin{cases} 1/L_a & \text{if edge } (i, j) \text{ belongs to found tour } V_a \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

where L_a is the cost of the a-th ant tour.

When an ant travels along a given path, this traveling takes an amount of time proportional to the travel distance (assuming that the ants move with constant speed). As pheromone is volatile, if a real ant travels more, the pheromone will have more time to evaporate, thus favoring better solutions to be discovered in the future. We conclude that adding pheromone evaporation to our model can be useful, especially for solving a complex problem like the TSP.

When an ant completes a tour, it will trace back its steps marking the edges on the way with pheromone. The update also takes into account pheromone evaporation. In our experiments, we used the rules of the ACS system [1], although our system is able to support other evaporation schemes. Evaporation and pheromone updates are implemented in ACS as follows:

$$\tau_{i,j} = (1 - \rho)\tau_{i,j} + \rho\Delta\tau^a_{i,j}, \tag{3}$$

where ρ is the evaporation rate: $0 \leq \rho < 1$.

All ants use Eq. (1) to probabilistically determine their next step. Therefore they will often choose the edge with the highest pheromone concentration, i.e., the exploration of less probable edges is low. The solution to increase the exploration of alternative paths is to remove some pheromone on edges chosen by ants, i.e., apply a local evaporation process. This has the effect of making them less desirable, increasing the exploration of the edges that have not been picked yet. So, whenever an ant travels along an edge, it applies local evaporation by updating the pheromone as follows:

$$\tau_{i,j} = (1 - \xi)\tau_{i,j} + \xi\tau_0, \tag{4}$$

where

- ξ is the local evaporation rate $0 \leq \xi < 1$, and
- τ_0 is the initial amount of pheromone on each edge.

A good heuristics to initialize pheromone trails is to set them to a value that is slightly higher than the expected amount of pheromone deposited by the ants in one tour; a rough estimate of this value can be obtained by setting $\tau_0 = 1/(nC)$, where n is the number of nodes, and C is the tour cost generated by a reasonable tour approximation procedure [1]. For example, we can set $C = nw_{avg}$, where w_{avg} is the average edge cost.

```
RECEIVE-ANT()
1. RECEIVE(ant)
2. ADJUST-ATTRIBUTES(ant)
3. SEND-TO(ant,BEST-NEIGHBOR(ant))

LOCAL-MOVE()
1. DEQUEUE(ant)
2. ADJUST-ATTRIBUTES(ant)
3. SEND-TO(ant,BEST-NEIGHBOR(ant))

SEND-TO(ant,destination)
1. if LOCAL(destination) then
2.    ENQUEUE(ant)
3. else
4.    SEND(ant,destination)

BEST-NEIGHBOR(ant)
1. if RETURNING(ant) then
2.    if BEST-TOUR(ant) then
3.       DEPOSIT-PHEROMONE()
4.    else return ANTHILL(ant)
5.    return LAST-VISITED-NODE(ant)
6. bestNeighbor ← RANDOM-CHOICE()
7. UPDATE-CRT-PATH-COST(bestNeighbor)
8. LOCAL-EVAPORATE-PHEROMONE()
9. ADD-TO-VISITED-LIST(bestNeighbor,ant)
10. return bestNeighbor

ADJUST-ATTRIBUTES(ant)
1. if AT-ANTHILL(ant) then
2.    if RETURNING(ant) then
3.       INITIALIZE(ant)
4.    else
5.          SET-RETURN-FLAG(ant)
6.          CALC-PHEROMONE-STRENGTH(ant)
7. UPDATE-BEST-TOUR()
```

Fig. 1. Algorithms for processing ant information.

3. Architecture

In ACODA, the ants' environment (i.e., the nodes of the graph representing the TSP map in our case) is designed as a set of interconnected software agents [10]. For the purpose of this work, by software agent (agent in what follows) we understand a software entity that (i) has its own thread of control and can decide autonomously if and when to perform a given action; (ii) communicates with other agents by asynchronous message passing; and (iii) can be located on an arbitrary machine in a computer network, providing that a certain runtime environment is locally available. Each agent can be referenced using its name, also known as the agent identifier.

The activity carried out by an agent is represented as a set of behaviors. A behavior is a sequence of primitive actions. Behaviors are executed in parallel with interleaving of actions on the agent's thread using a non-preemptive scheduler that is internally available to each agent. Additionally, each agent maintains a message queue that holds all its incoming messages [10].

Ants are software objects exchanged by nodes. We configured ACODA such that each agent manages a part (called the submap) of the initial TSP map. A submap contains several nodes, and the TSP map is partitioned as a set of submaps. Ants can move between local nodes (nodes managed by the same agent) as well as between nodes that are managed by different agents, usually located on different machines. In the latter case, an ant move is more costly by sending a message from the agent that manages the source node to the agent that manages the destination node.

A sample configuration of ACODA for a partitioned TSP map is shown in Fig. 2. One can easily notice that the map contains $n = 12$ nodes, there are $k = 4$ computers and $N = 6$ software agents, and on each computer we allocated $l = 3$ nodes. Ants can migrate between computers (for example the ant moving from node 4 on computer C_2 to node 3 on computer C_1 via a message exchange between agents A_3 and A_2) or between nodes residing on the same computer (for example the ant moving from node 7 to node 8 on computer C_4 via a message exchange between agents A_6 and A_5 or from node 10 to node 11 on computer C_3 without any message exchange between agents because both nodes are managed by agent A_4).

Ants have a set of attributes: the cost of the currently explored path, pheromone strength, returning flag, best tour cost (value of the currently best tour that the ant knows, based on its search history) and a list of nodes representing the path that the ant followed to reach its current location. The list is necessary for two reasons: (i) the ant needs to trace back its steps in order to mark the tour with pheromone and (ii) we need to avoid loops so only unvisited nodes are taken into account as possible next hops. Attributes are initialized when an ant is created. Subsequently, they are updated during the process of ant migration in order to reflect the current knowledge of the ant about the explored environment. For example, whenever an ant completes a tour by reaching its destination node (its anthill), the agent that is managing that node saves locally the ant's currently best tour. Whenever another ant from the colony visits this node, it is able to sense the environment and eventually to update its "current knowledge" about the value of the best tour found so far. Hence ants have the possibility to exchange via the environment (the set of agents that manage the map nodes) not only pheromone information, but also information about the best solutions found so far.

Each computer runs one or more software agents that control the ant movement. Agent design must include behaviors for correctly transferring the ants between nodes. Whenever an agent receives an ant, its RECEIVE-ANT() behavior (see Fig. 1) updates it and then sends it to a neighbor node following ACO rules. Ants that must move to local nodes will be pushed onto the local ant queue (there is one such queue for each agent), while the other ants are sent out as messages to the agent that manages the destination node. The behavior LOCAL-MOVE() continually pops the next ant from the local queue and sends it out to a neighbor node following ACO rules.

Each agent holds the following set of internal parameters: pheromone evaporation coefficient, best tour cost (the value of the currently best tour that the agent knows, based on ants that traveled through this agent), a set of lists of neighbor nodes (there is a list for each node managed by this agent) including the weights and the values of pheromone deposited on the corresponding edges, and a hash map that associates to each node the identifier of its managing agent. Each agent and

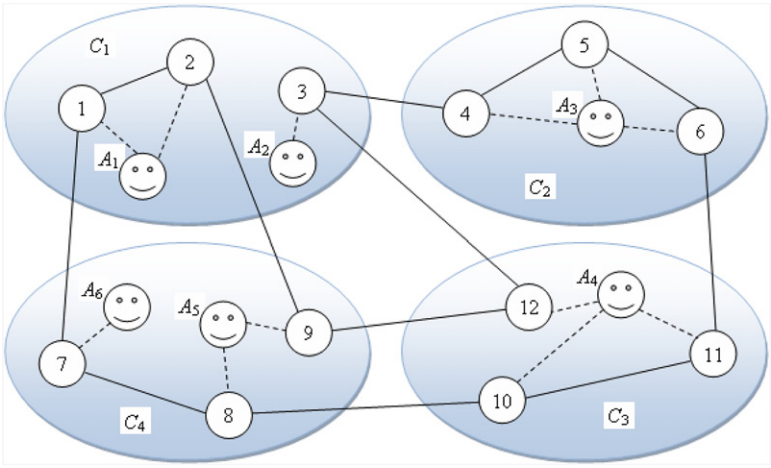


Fig. 2. ACODA sample configuration.

each ant maintain their own values of the currently best tour encountered so far. So, whenever an ant reaches a node, the node's manager agent can update its best tour information.

Each agent creates its own ant population for each node that it manages. The agents calculate pheromone strength according to the tour cost and also update the ants' pheromone strength attribute. Additionally, they set the returning flag for ants that have completed a tour, exchange ant information with other agents, deposit pheromone when needed, and update the cost of the current search path of an ant by adding the weight of the chosen edge.

The functionality of an agent is described as follows as a set of behaviors. The RECEIVE-ANT() behavior retrieves an ant message from the agent's message queue (if this queue is not empty), adjusts the ant's attributes using the ADJUST-ATTRIBUTES() method and sends it to the node determined by the BEST-NEIGHBOR() method (this could be either a local node or a node external to this agent). The ADJUST-ATTRIBUTES() method sets a returning flag and calculates the pheromone strength whenever an ant has completed a tour. Ants that have returned to the anthill are re-initialized.

BEST-NEIGHBOR() determines the node where an ant should be sent. When the ant returns to the anthill, this method sends the ant to the first node of its list of visited nodes, popping it from the list, and then deposits the ant's pheromone on the corresponding edge. This method also implements pheromone evaporation. The SEND-TO() method either sends out an ant to the agent that manages the destination node, or pushes the ant onto the agent's local ant queue if the destination node is managed by the current agent. The LOCAL-MOVE() behavior pops an ant from the local queue, adjusts the ant's attributes using the ADJUST-ATTRIBUTES() method and then sends it out to the node determined by the BEST-NEIGHBOR() method using Eq. (1). AT-ANTHILL() returns true if the current node is the anthill of the ant given as parameter. ANTHILL() returns the identifier of the agent that manages the anthill of the ant given as parameter.

Although the architecture looks quite biased towards TSP solving, it can be adapted to solve other optimization problems, including the resource location problem, the shortest path problem, generalized TSP and other graph problems by overriding the methods ADJUST-ATTRIBUTES() and BEST-NEIGHBOR(). The ACO algorithm is also configurable by overriding the methods BEST-NEIGHBOR(), ADJUST-ATTRIBUTES(), LOCAL-EVAPORATE-PHEROMONE() and DEPOSIT-PHEROMONE().

4. Experiments

We created a distributed platform based on a JADE multi-agent framework [10] that facilitates experimentation with ACODA. The platform can be configured to run distributed ACO algorithms on an arbitrary computer network. The focus of our experiments was to perform an initial analysis of the scalability of ACODA in order to support the feasibility of our approach to distributed ACO. We used the platform to run ACODA on a computer cluster, as well as on a high-speed computer network, by varying the number of available machines, as well as the size of the benchmark TSP.

We start this section with the general principles for the experimental setup of ACODA that cover the description of an experiment, the parameters of an experiment, and the procedure for setting up and running ACODA, as well as the performance measures recorded in our experiments.

4.1. Experimental setup

An experiment is organized as a fixed number of independent experimental rounds. A round consists of one execution of ACODA for a given set of parameters and it ends immediately after a predefined number of ants (given as a parameter) are received by a designated node of the map. This node is called the *reference node*, and it is given as an input parameter. All parameters are initialized at the beginning of each round. Experimental data are collected during each round. At the end of the experiment these data are post-processed to calculate performance measures.

We use the following set of parameters for configuring ACODA.

- k, the number of computers (machines or cluster cores).
- n, the number of nodes of a TSP map. The nodes of a TSP map are identified by numbers from 0 to $n-1$ according to [13].
- M, the maximum number of movements of an ant. We define an ant movement as the action of transferring the ant from a source node to a destination node along a given edge.
- *Master*, the agent that manages the reference node.
- N, the number of agents.
- r, the number of experimental rounds of an experiment; we have set $r = 10$ in all the experiments described in this section.

Additionally, the following information must be defined in order to specify a complete ACODA configuration:

- The reference node which controls the termination of an experimental round by counting a given number of received ants (set to M[1]).
- The policy of mapping nodes to agents as well as of allocating agents to computers. Actually in our experiments we have used two policies: (i) a simple policy in which we allocate a single agent to each available computer and then we distribute evenly the nodes to agents (in this case $N = k$), and (ii) a policy which allocates a given fixed number $l \geq 1$ of nodes per agent and then distributes the agents evenly onto the k available computers (in this case $N \geq k$).
- The initial populations of ants for each node. Each initial population can contain zero, one, or more ants. Although this parameter is configurable, in this study we considered only initial populations consisting of one ant for each node of the TSP map. Basically this means that in each experiment we have exactly n ants.

The ACO parameters were set to the values recommended by [1] for the ACS on which our experiments were based: $\tau_0 = 1/(n^2 w_{avg})$, $\rho = \xi = 0.1$, $\alpha = 1$, $\beta = 5$. The number M of moves of each ant was chosen according to the proposal from [1]. There, 1000 moves per ant were chosen for a map of 19 nodes. Taking into account the sizes of the maps that we used in our experiments (varying between 51 and 666) and scaling up accordingly, we chose a number $M = 10\,000$ of moves per ant.

Setting up and running ACODA on an arbitrary computer network assumes two stages: (i) the initialization stage and (ii) the execution stage.

4.1.1. Initialization stage

During the initialization stage, the following steps are performed: the JADE platform is started, a number of JADE agent containers are created (one container for each available machine), the nodes are prepared to be evenly assigned to agents, and finally the agents are created and evenly distributed on available containers of the JADE platform. As we were using a single agent for each network machine, in our setup we also created a single JADE container on each available machine.

Note that our architecture can easily support more agents running on each machine, to better exploit its local computing resources. Moreover, it is not necessary to run the same number of agents on each machine. Rather, this number can be tailored to optimize the usage of each machine's computational resources if ACODA is configured and deployed on a heterogenous computer network. Note also that the use of multiple JADE containers on the same machine is not advisable, because they would waste the machine resources.

4.1.2. Execution stage

The ACO algorithm is run during the execution stage. Execution is controlled using special control messages. Control messages have their type set to "command", while ant exchange messages have their type set to "ant". Command messages are given higher priority than ant exchanging messages.

The experimental rounds are automatically started and stopped to avoid the need for repeated executions of the run script. An experimental round starts when all agents receive a "start" command, and ends when all agents receive a "stop" command. A designated agent called *Master*, which manages the reference node, is responsible for controlling the experimental rounds by issuing "start" and "stop" commands and calculating performance measures. The *Master* counts the total number of ants that were received by the reference node. When this number reaches a given maximum value M, *Master* stops the current round and starts a new fresh round. At the beginning of a round, each node must generate its initial population of ants, which in our experiments with the TSP was set to a single ant for each node.

4.1.3. Performance measures

The duration T_i of an experimental round i is recorded by the *Master* agent as time elapsed between issuing a "start" and "stop" command. While the distributed ACO algorithm is running, minimum tour costs are passed from agent to agent via ants. Whenever an agent updates its current value of the minimum tour, the time elapsed since the agent received the

[1] When the reference node counted M received ants, taking into account that on the average one reception means one complete tour, we get a total number of $n \times M$ ant movements. However, if we assume that the initial population of ants of each node consists of a single ant, it means that there are exactly n ants in the system. This simple reasoning shows that indeed this means approximately M moves per ant.

Table 1
Calculation of performance measures.

$v_i = \min_{j=0}^{n-1} v_{i,j}$	v_i is the solution found by round i.
$t_i = \min_{j=0}^{n-1}\{t_{i,j} \mid v_{i,j} = v_i\}$	t_i is the time in which solution was found by round i.
$v_{avg} = (\sum_{i=1}^{r} v_i)/r$	v_{avg} is the average value of solutions found in all rounds.
$t_{avg} = (\sum_{i=1}^{r} t_i)/r$	t_{avg} is the average time in which solutions were found in all rounds.
$v_{min} = \min_{i=1}^{r} v_i$	v_{min} is the best solution found after carrying out all rounds.
$t_{min} = \min_{i=1}^{r}\{t_i \mid v_i = v_{min}\}$	t_{min} is the minimum time in which the best solution was found in all rounds; it indicates when the best solution was first found.
T_i	T_i is the duration of experimental round i.
$T_{avg} = (\sum_{i=1}^{r} T_i)/r$	T_{avg} is the average execution time of the rounds in an experiment.

"start" command is also recorded. When an experimental round is finished, the *Master* agent saves the experimental data collected during the round. Note that this data collection does not affect the distributed nature of the architecture since it is done solely for experimental analysis.

Note that we cannot assume that the best tour recorded by the *Master* is actually the best tour computed during an experimental round. So we need to determine the minimum of the best tours computed by all agents together with the time at which this tour was discovered.

We utilize the performance measures introduced in [7]. Let us suppose that we have n nodes and r experimental rounds. For each node j, let $t_{i,j}$ be the time of the last update of the best tour performed by agent j in round i, and let $v_{i,j}$ be the cost of the corresponding tour. The *Master* agent will collect values $v_{i,j}$ and $t_{i,j}$, as well as determine the solution v_i and the associated time t_i in round i, as shown in Table 1. The experimental data that were acquired during all the rounds of an experiment are post-processed to calculate performance measures, as shown in Table 1.

4.2. Experiments on a cluster with workload management system

In this experiment, ACODA was deployed and tested on an Infragrid cluster consisting of 128 cores. Each core is equipped with 1 GB of RAM and all the cores are interconnected using an Infiniband 40 Gbits/s network. The cores share an external storage space. This computer cluster is composed of 16 servers with 8 cores each of type Intel Xeon CPU E5504. Each server is allocated a single IP address. The cluster is running Linux and the Condor workload management system [16]. Condor provides a job queueing mechanism, scheduling policy, priority scheme, resource monitoring, and resource management. This allows users to submit their parallel or serial jobs, which are scheduled for execution such that to optimize the usage of cluster resources. Submission of a job is done by means of a Condor script that specifies the input, output and Condor log files, the executable and the universe. A Condor universe defines an execution environment. Note that the Condor software allows one to use each separate core as a machine of an ordinary computer network, so we shall use the term machine instead of core in what follows.

For the purpose of this experiment, we allocated one agent for each network machine. Let q be the quotient and p the remainder of the division of n by k. Clearly $p < k$, so the value $k - p$ is positive. For the assignment of nodes to agents we use the default ordering of node identifiers: we start with node 0 and we assign each of the following $q+b$ nodes to an additional agent, starting with node 0. In this distribution scheme, $b = 1$ for the first p agents and $b = 0$ for the remaining $k - p$ agents. The correctness of this allocation trivially follows from the equality $n = k * q + p = (k - p) * q + p * (q + 1)$. Therefore each agent manages either q or $q + 1$ nodes. Nodes assigned to each agent are ordered according to their identifiers. For example, if $n = 51$ and $k = 2$, the first agent will manage 26 nodes numbered 0 to 25, while the second agent will manage 25 nodes numbered 26 to 50.

4.2.1. Initialization stage

This stage was a challenge in itself, since the *Java* universe provided by Condor supports only serial jobs. Thus we had to use the *Parallel* universe to run a shell script on multiple machines. The shell script executes ACODA as a Java library. This library is used to start the platform as well as the required containers depending on whether the first argument is "platform" or "container". The second argument indicates the name of the file containing the TSP map and the third is the number of experimental rounds. Only the platform needs to be provided with these additional arguments. The arguments are needed to create and initialize a bootstrap agent that is only responsible with the initialization and starting of the experiment according to the standard procedure. When the ACODA library is run with the "container" argument, it will read the "APDescription.txt" file from the shared storage space. This file is automatically created when JADE is started, and it helps to identify the platform and its location in the cluster.

4.2.2. Execution stage

Taking advantage of a larger number of machines available for this experiment, we have set the goal to determine the number of machines for which the average execution time T_{avg} of the $gr666$ problem is minimum. Intuitively, if the number

Table 2
Experimental results on *gr*666 [13] with an optimum tour cost of 294 358.

cpu	v_{avg}	t_{avg} [s]	v_{min}	t_{min} [s]	T_{avg} [s]
1	337 931	3229	331 038	1122	8917
7	336 546.4	53.8	333 160	69.1	82.9
10	335 771.6	27.1	333 180	16.4	43.7
11	335 180.4	16.6	329 184	18.8	40
12	336 367.2	27.4	333 176	17.1	48
16	343 816	489.2	333 716	457	559.5

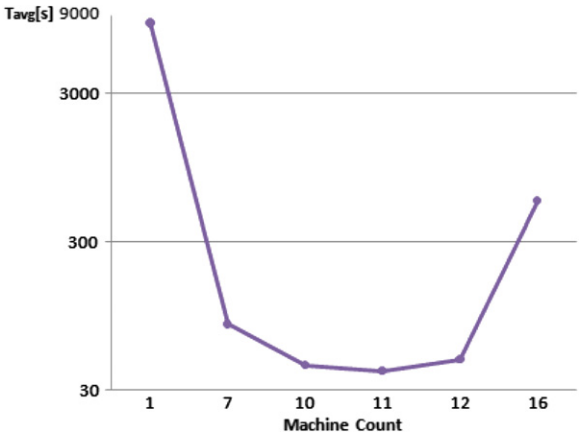


Fig. 3. Graphical representation of T_{avg} in relation to machine count.

of machines were too large, the communication overhead would become very high, while if the number of machines were small, the machines (i.e., agents, as we assume one agent per machine) would spend too much time with local movements of ants.

Table 2 shows the results of the experiment conducted on the map *gr*666 from the TSP benchmark library [13]. We ran several experiments, of 10 experimental rounds each, on up to 16 machines from the cluster. The goal of the experiment was to find the number of machines for which the execution time T_{avg} is minimum. We ran the control experiment on one machine and recorded all performance measures. We then started the scalability test by increasing the number of machines to 7. Next we significantly increased the number of machines, and consequently observed larger execution times. We determined the minimum T_{avg} through a sequence of tests for different numbers of machines. The execution time T_{avg} was largest for a single machine and smallest for 11 machines. Intuitively, for each given input data set, a distributed architecture (including ACODA) has a limit to how much it can scale. In the case of our experiment for *gr*666, ACODA could only scale up to 11 machines. By using more machines, the communication becomes too time consuming, increasing the overall execution time.

In Fig. 3, we present a graphical illustration of T_{avg} depending on the number of machines used in each experiment. Please note that for visualization purposes we used a logarithmic Y scale, so the differences are much larger than they appear in that figure.

As the cluster we have used for running the experiments provides 128 machines, we hope to be able to run experiments with good results on TSP problems larger than *gr*666. However, this was left as future work.

4.3. Experiments on a cluster without a workload management system

In this experiment, we used an ordinary computer network of 7 machines equipped with dual core processors at 2.5 GHz and 1 GB of RAM memory. These machines were interconnected using a high-speed Myrinet[2] interconnection network at 2 Gb/s. We ran the experiment on $k \in \{1, 4, 7\}$ computers. For each computer network configuration we ran three tests with variable number of nodes $l \in \{1, 5, 10\}$ that are managed by each agent in the system. Consequently, in this experiment, we created multiple agents on each computer of the network. Note that, as n is not always a multiple of l, actually it is not possible that all the agents will manage exactly l nodes.

The policy for allocating nodes to agents is defined as follows. Let q and s be the quotient and the remainder of the integer division of n by l. Obviously $0 \leq s < l$ and $n = l * q + s$. We distinguish between two cases. If $s < q$, then we set the number

2 http://www.myri.com/myrinet/overview/.

Table 3
Experimental results. The number in the map name is the number of nodes and the value written between brackets is the optimum tour cost. Speedup values are calculated as the ratio of T_{avg} for one node per agent on one computer and T_{avg} of the current row.

Map	k	l	v_{avg}	t_{avg} [s]	v_{min}	t_{min} [s]	T_{avg} [s]	Speedup
eil51 (426)	1	1	459.2	33	452	27	79	1
	1	5	489.6	18	478	20	57	1.3
	1	10	480	16	475	18	53	1.4
	4	1	462.4	46	449	64	60	1.3
	4	5	485.8	18	479	16	42	1.8
	4	10	469.8	23	461	3.9	40	1.9
	7	1	449.4	13	442	10	39	2
	7	5	480.8	13	472	25	40	1.9
	5	10	471.6	15	467	17	37	2
st70 (675)	1	1	729.4	67	712	38	127	1
	1	5	786.6	16	772	41	80	1.5
	1	10	779	20	766	47	65	1.9
	4	1	720.4	32	701	45	90	1.4
	4	5	786.2	20	784	26	74	1.7
	4	10	777	24	757	55	70	1.8
	7	1	688.6	25	684	6.4	56	2.2
	7	5	752.3	20	702	17	47	2.7
	7	10	771.4	32	691	40	43	2.9
kroA100 (21282)	1	1	22 590.8	38	22 152	31	274	1
	1	5	22 443.8	37	21 847	136	168	1.6
	1	10	22 554.6	86.9	21 735	110	136	2
	4	1	22 677.8	22	22 232	15	145	1.9
	4	5	22 513.2	32	21 896	103	121	2.2
	4	10	22 625.2	84	21 930	115	120	2.3
	7	1	22 997	29	22 605	77	92	2.9
	7	5	22 141	20	21 748	16	80	3.4
	7	10	22 319.8	45	21 937	39	80	3.4
ch150 (6528)	1	1	7969.3	257	6992	524	572	1
	1	5	7572.6	139	6974	149	558	1
	1	10	7166	145	6922	167	380	1.5
	4	1	7650.8	135	7434	281	291	1.9
	4	5	7771	62	7013	98	227	2.5
	4	10	7503.7	57	7048	82	211	2.7
	7	1	7729.4	17	7638	26	194	2.9
	7	5	7601.8	37	6851	43	161	3.5
	7	10	7594.2	32	6965	52	155	3.7
gr666 (294358)	1	666	337931	3229	331038	1122	8917	1
	4	166	347440	688	341808	596	1920	4.6
	7	95	342352	398	339416	1268	1458	6.1

of agents to $N = q$ such that each of the first $q - s$ agents manages l nodes, and each of the remaining s agents manages $l + 1$ nodes. The correctness of the allocation follows from the equality $n = l * q + s = l * (q - s) + (l + 1) * s$. If $s \geq q$, then we set the number of agents to $N = q + 1$ such that each of the first s agents manages l nodes and the last one manages s nodes such that $l > s \geq q$. For example, if $n = 100$ and $l = 13$, then $q = 7$ and $s = 9$. Because $q < s$, we set $N = 8$ agents. Each of the first 7 agents manages 13 nodes, while the last agent manages 9 nodes.

On this smaller cluster, we did not have the advantage of a workload management system, so we could not use the initialization and execution script. However, the significantly smaller size of the network permitted us to manually start JADE on each computer and then run the bootstrap agent.

We ran several experiments using the following benchmark TSP maps selected from TSPLIB [13]: *eil*51, *st*70, *kroA*100, *ch*150, and *gr*666. The number that occurs in the map name indicates the number n of map nodes. These maps were chosen to experiment with different values of w_{avg} and $\Delta w = w_{max} - w_{min}$, where Δw is the difference between the maximum and the minimum edge weight and w_{avg} is the average edge weight.

The experimental results (see Table 3) show a significant improvement of the execution time when the number of nodes managed by each agent is increasing. As shown in this table, it is clear that the execution time T_{avg} diminishes when the number l of nodes per agent increases for every map, regardless of the number of the computers in use. For example, in the case of map *eil*51, when running on a single computer, we obtained $T_{avg} = 79$ s for $l = 1$ and $T_{avg} = 53$ s for $l = 10$.

Note that by including experiments with $l = 1$, i.e., one node per agent, we are comparing the results of the map partitioning approach with the version without map partitioning [7]. While clearly map partitioning is advantageous, the experiments (see Table 3) also show that increasing the number of computers determines the decrease of the average

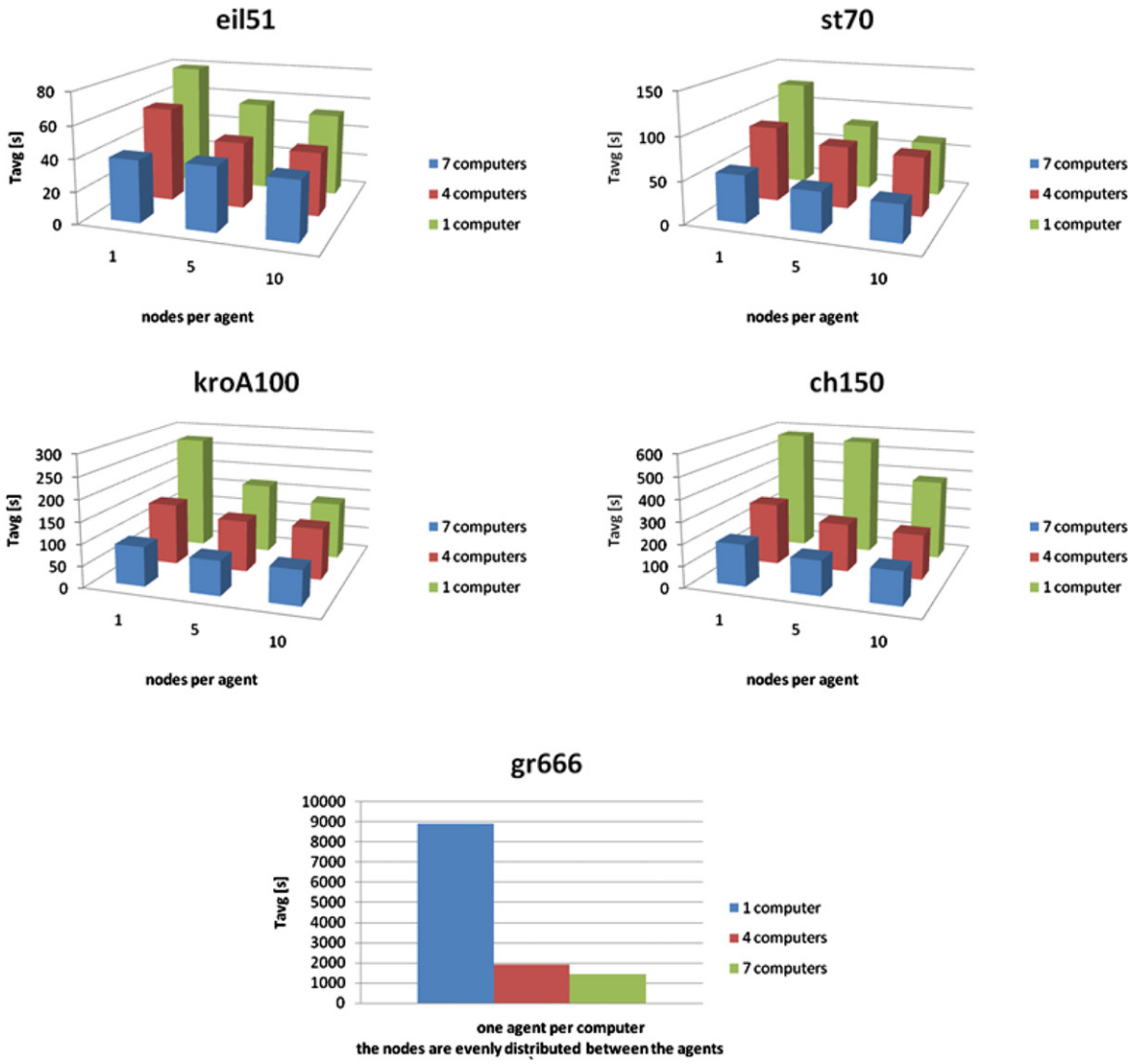


Fig. 4. Graphs of the execution time T_{avg}.

execution time of the algorithm, so the scalability of ACODA is preserved when each ACODA agent is managing more nodes. For example, in the case of problem *kroA*100 when $l = 5$, T_{avg} decreases from 168 s when a single computer is used to 80 s when 7 computers are used, and similar conclusions are drawn for $l = 10$, i.e., a decrease from 136 s, for 1 computer, to 80 s, for 7 computers.

The speedup values shown in the last column of Table 3 are calculated as the ratio of (i) the value of T_{avg} for the configuration with one node per agent and a single computer and (ii) the value of T_{avg} shown on the current row of the table. These results clearly show that when the number of computers as well as the number of nodes per agent are increased we obtain better speedups (see also Fig. 4). The experiments also show that this method of improving efficiency is even more effective on larger maps. For example, for map *eil*51, we could only obtain a speedup of 2 at most, while on larger maps such as *ch*150 and *kroA*100 we could obtain speedup values above 3.

It is worth mentioning that in both experiments reported in Sections 4.2 and 4.3 we did not employ any special improvements of the underlying ACS. Therefore, we do not comment here on the quality of the solutions. We did, however, include the obtained tour lengths in order to show that our approach does not affect the quality of the solutions as found by the bare ACO approaches (i.e., no specialized improvements, such as local searches before marking the paths with pheromone, were used). The application of ACODA to more sophisticated forms of ACO was left as a future investigation path.

5. Related work

Although the TSP is a classic benchmark problem for heuristic search algorithms, based on our literature review we found very few works that propose ACO-based distributed TSP algorithms that utilize recent advances of multi-agent system

middleware. Nevertheless, we could find references to multi-agent approaches to ACO for other combinatorial optimization problems.

A related approach to agent-based distributed ACO is presented in [2]. There, graph nodes and ants are implemented as JADE agents. Ants send to a central place the information about pheromone deposits and nodes' best tour cost through a single message exchange per node [10]. This procedure adds up to $2n$ messages per tour per ant, where n is the number of nodes. Each ant has to notify the node about its next hop and the cost of its tour in order for the node to be able to update its pheromone levels. This generates another n messages. When an ant completes a tour, it compares the tour cost with the collected best tours from the nodes. A best tour synchronization is triggered for all the nodes if a better tour has been found. This brings an additional overhead of n messages. So this approach requires $4n$ messages per tour per ant, while our approach requires $2n$ messages: n messages (ant movements) to complete a tour and n messages to deposit the pheromone. We avoid the additional overhead of sending to agents all the information necessary to carry out their tasks, as in ACODA this information is already contained in ant messages exchanged between nodes. Note also that in ACODA many messages are reduced to local enqueue–dequeue operations which are much faster than exchanging messages between agents.

The ACODA configuration reported here is based on the sequential ACS presented in [1]. There are however three notable differences. (i) We avoid iterations which are specific to the classic sequential formulation of the ACO algorithms. An iteration lasts until every ant has found a tour. It would be time consuming to synchronize all agents by waiting until all the ants finalize their tours. This would cancel the main benefits of an asynchronous, distributed architecture. (ii) In ACS, the best tours are compared after each iteration, thus allowing only the best ant to mark its tour. Again, this would require synchronization and centralization, so we avoid it by allowing all ants to mark their tours. (iii) In ACS, ants move synchronously, taking one step at a time, while in our approach ants move asynchronously.

Paper [3] propose JABAT (JADE-based A-team), a JADE-based middleware for collaborative problem solving using asynchronous teams known as A-teams. JABAT supports distributed implementation and collaboration of population-based optimization algorithms. In particular, JABAT was applied to the TSP. There is however a very important difference between JABAT and ACODA. JABAT agents represent sequential improvement algorithms that cooperate for solving a given problem in a distributed way. ACODA agents provide a natural distributed agent-based model of the problem environment that is suitable for distributed ACO algorithms. Moreover, we could not find scalability studies of the JABAT distributed architecture.

[17] proposes a JADE-based multi-agent environment for dynamic manufacturing scheduling that combines intelligent techniques of ACO and multi-agent coordination. However, the focus in [17] is to evaluate the impact of ACO intelligence on multi-agent coordination, rather than to utilize multi-agent middleware for improving the efficiency of ACO. Therefore ACO intelligence is embedded into job and machine agents that have the role of ants, which is different from ACODA, where ants are passive objects exchanged by agents providing a distributed model of the problem environment.

In [18], the authors compare a distributed form of ACS with the flooding algorithm applied to resource discovery problem using the *ns-2* network simulation tool [19]. They show that ACS is the better approach in terms of best success rate, least number of hops, and least traffic. However, we were not able to duplicate their approach using the ACODA framework for an accurate comparison because the details of their system, algorithm, and ACO parameters were not available in their paper. The differences between their approach and ACODA are that (i) resource queries are handled centrally at a single anthill, thus introducing a single point of failure, (ii) they do not take edge weights into account as they are trying to solve the resource discovery problem, not the TSP, and (iii) ants are implemented as *ns-2* mobile agents, while in ACODA we are using JADE. Moreover, in practice very often there is no need for code mobility as every ant is governed by the same behavioral rules. So, in our approach, we use the paradigm of "agent-managed nodes" to control ants' movement using message exchanges between agents, rather than using code (i.e., agent) mobility.

[20] introduces a theoretical framework for self-organizing coordination-and-control applications using multi-agent systems (MASs). In particular, the authors introduce three application patterns: *Smart messages*, *Delegate MAS* and *Delegate ant MAS*. The patterns are capable of implementing distributed ACO. *Smart messages* are entities encapsulating state and behavior similarly to mobile agents, and they can be used to implement complex interactions with nodes of a dynamic network. A *Delegate MAS* is a complex system that manages a collectivity of smart messages to solve a given task. Finally, the *Delegate ant MAS* is a specific type of *Delegate MAS* that is capable of implementing distributed ACO behaviors. The authors do not address the problem of representing the environment using MAS and they do not go in the details of ACO including the convergence, stopping condition, and evaluation of ACO based on their proposals.

In [4], the authors thoroughly analyze multi-colony ACO applied on the TSP. The main difference from our approach is that [4] proposes running a separate colony on each available processor. At predetermined time points, the solutions are centrally collected and a local search is performed on the best solution provided by each colony. This distributed ACO approach is not fully decentralized, as it requires synchronization and centralized sequential local search after each solution is centrally collected. Our approach is fully decentralized, parallel and asynchronous, thus leading to heavy parallelization.

In [21], the authors present a neural network approach for optimal partitioning of TSP maps. The idea was to improve efficiency by choosing an optimal partitioning of the TSP map such that combining the tours for each submap of the partition will result in a nearly optimal cost tour. The differences from our approach are that (i) our approach is distributed, while no attempt for distribution is made in [21]; (ii) whatever initial arbitrary partitioning scheme we use, our algorithm can still find good results as our ants are not restricted to finding optimal tours for each submap of the partition; rather, they can freely move from one submap to another; and (iii) we use ACO on the whole map, rather than on each individual submap of the partition.

A similar idea to our map partitioning approach was employed by the scalable distributed constraint architecture called DisChoco that is described in [22]. In this approach, similarly to our proposal, agents manage parts of the problem domain as well as the constraints specific to their partition. The agents propose, propagate, or refuse partial solutions to each other, exactly as ACODA agents update and exchange ant information.

A very recent overview and classification of parallel computing approaches to ACO was reported by [6]. The authors propose an interesting and novel classification scheme for parallel ACO algorithms. The classification includes a master–slave model, cellular model, parallel independent runs model, multi-colony model (also addressed by [4]), and hybrid models. It is interesting to note that according to the cellular model the ant colony is structured as a set of small and overlapping neighborhoods. We found that this model is the closest to our approach of the partitioned map. The similarity comes from the usage of a distributed memory implementation of the cellular model, while in ACODA the pheromone matrix can be also seen as distributed among the set of agents that manage the TSP map nodes. The main differences between the two approaches are that (i) while we have applied ACODA to solve the TSP, the authors of [6] applied ACO to solve the generalized Steiner problem, and (ii) in ACODA we follow the "standard" proposal of ACO (in particular the ACS system), while [6] employs the diffusion model to spread the effect of finding good solutions beyond the neighborhoods of each ant. Nevertheless, it would be interesting as future work to compare ACODA with the cellular model on the same optimization problem.

6. Conclusions and future work

In this paper, we have presented a configurable distributed architecture called ACODA that provides a novel and more intuitive way of distributing ant-based nature-inspired computational processes onto state-of-the-art multi-agent middleware. Moreover, we realized an initial experimental evaluation of this architecture by applying it to solve the TSP. The main innovations introduced by our approach are that (i) the physical environment of the ants is conceptualized, represented, and implemented as a distributed multi-agent system, and (ii) ant management that is responsible for ant migration in the physical environment is represented as messages exchanged asynchronously between the agents of the problem environment. ACODA can be adapted to support various ACO algorithms. Additionally, it can be configured to run on computer clusters, as well as on ordinary computer networks. The initial experimental results support the feasibility and scalability of our architecture. As future work, we intend to strengthen the experimental results by applying ACODA to larger TSP problems, as well as to other optimization problems, on larger computer networks. Moreover, we are interested in comparing ACODA with other parallel ACO approaches reported in the literature. Among them, the most interesting from the point of view of evaluating the distributed multi-agent architecture on which ACODA is based are the multi-colony model and the cellular model.

Acknowledgements

This work was partially supported by the strategic grant POSDRU/88/1.5/S/50783, Project ID50783 (2009), co-financed by the European Social Fund – Investing in People, within the Sectoral Operational Programme Human Resources Development 2007-2013.

We thank the members of HPC Center of West University of Timişoara, Romania[3], for allowing us to run our experiments on their computer cluster.

References

[1] M. Dorigo, T. Stutzle, Ant Colony Optimization, MIT Press, 2004.
[2] E. Ridge, D. Kudenko, D. Kazakov, Parallel, asynchronous and decentralised ant colony system, in: Proc. of the First International Symposium on Nature-Inspired Systems for Parallel, Asynchronous and Decentralised Environments (NISPADE), 2006.
[3] I. Czarnowski, P. Jedrzejowicz, I. Wierzbowska, A-team middleware on a cluster, in: Proc. 3rd KES International Symposium on Agent and Multi-Agent Systems: Technologies and Applications: KES-AMSTA'2009, Vol. 5559, Springer-Verlag, 2009, pp. 764–772.
[4] C. Twomey, T. Stützle, M. Dorigo, M. Manfrin, M. Birattari, An analysis of communication policies for homogeneous multi-colony ACO algorithms, Information Sciences 180 (12) (2010) 2390–2404.
[5] A. Al-Dahoud, A.M. Belal, M.B. Al-Zoubi, Load balancing of distributed systems based on multiple ant colonies optimization, American Journal of Applied Sciences 3 (7) (2010) 428–433.
[6] M. Pedemonte, S. Nesmachnow, H. Cancela, A survey on parallel ant colony optimization, Applied Soft Computing (in press), Corrected Proof (2011).
[7] S. Ilie, C. Bădică, Distributed multi-agent system for solving traveling salesman problem using ant colony optimization, in: M. Essaaidi, M. Malgeri, C. Bădică (Eds.), Intelligent Distributed Computing IV, in: Studies in Computational Intelligence, vol. 315, Springer, Berlin/Heidelberg, 2010, pp. 119–129.
[8] S. Ilie, A. Bădică, C. Bădică, Distributed agent-based ant colony optimization for solving traveling salesman problem on a partitioned map, in: Proceedings of the International Conference on Web Intelligence, Mining and Semantics, WIMS '11, ACM, 2011, 23:1–23:9.
[9] S. Ilie, C. Bădică, Effectiveness of solving traveling salesman problem using ant colony optimization on distributed multi-agent middleware, in: Proceedings of the International Multiconference on Computer Science and Information Technology, 2010, pp. 197–203.
[10] F.L. Bellifemine, G. Caire, D. Greenwood, Developing Multi-Agent Systems with JADE, John Wiley & Sons Ltd, 2007.
[11] S. Depickere, D. Fresneau, J. Deneubourg, Effect of social and environmental factors on ant aggregation: A general response? Journal of Insect Physiology 54 (9) (2008) 1349–1355.

[3] http://hpc.uvt.ro.

[12] C. Detrain, J.-L. Deneubourg, Self-organised structures in a superorganism: do ants behave like molecules? Physics of Life Reviews 3 (3) (2006) 162–187.
[13] G. Reinelt, Tsplib — a traveling salesman library, ORSA Journal on Computing 3 (1991) 376–384.
[14] C. Chira, C.-M. Pintea, D. Dumitrescu, Learning sensitive stigmergic agents for solving complex problems, Computing and Informatics 29 (3) (2010) 1001–1020.
[15] M. Wooldridge, An Introduction to MultiAgent Systems, John Wiley & Sons Ltd, 2002.
[16] D. Thain, T. Tannenbaum, M. Livny, Distributed computing in practice: the condor experience, Concurrency and Computation: Practice and Experience 17 (2–4) (2005) 323–356.
[17] W. Xiang, H.P. Lee, Ant colony intelligence in multi-agent dynamic manufacturing scheduling, Engineering Applications of Artificial Intelligence 21 (1) (2008) 73–85.
[18] S.M. Fattahi, N.M. Charkari, Ant distributed ACS algorithm for resource discovery in grid, International Journal of the Computer, the Internet and Management 17 (SP1) (2009).
[19] Teerawat Issariyakul, Hossain Ekram, Introduction to Network Simulator NS2, Springer, 2009.
[20] T. Holvoet, D. Weyns, P. Valckenaers, Patterns of delegate mas, in: Third IEEE International Conference on Self-Adaptive and Self-Organizing Systems, SASO'2009, IEEE Computer Society, Los Alamitos, CA, USA, 2009, pp. 1–9.
[21] N. Aras, I.K. Altinel, J. Oommen, A Kohonen-like decomposition method for the euclidean traveling salesman problem – knies_decompose, IEEE Transactions on Neural Networks 14 (4) (2003) 869–890.
[22] R. Ezzahir, C. Bessiere, M. Belaissaoui, E. Bouyakhf, Dischoco: A platform for distributed constraint programming, in: Proceedings of the Eighth International Workshop on Distributed Constraint Reasoning at IJCAI'07, 2007.